UNIVERSIDAD EL BOSQUE



FACULTAD ING. DE SISTEMAS

BASES DE DATOS II

PROYECTO INTEGRADOR · AVANCE 1: NÚCLEO TRANSACCIONAL EN POSTGRESQL

EMPRESA DE CINE: ALTAMIRA CINE

INTEGRANTES
ESTEFANIA PAREDES CASTAÑEDA
JOSE ANTONIO ESPIN MOLINA
SANTIAGO ROJAS
ERIKA GOMEZ

TAMAÑO DEL GRUPO: 4 INTEGRANTES
DOCENTE: FRANCISCO BARGUIL VANEGAS

15 DE OCTUBRE DEL 2026

Datos de la entrega

Repositorio: https://github.com/tef333/Proyecto-integrador-cinema

Estructura: carpeta avance1-postgresql/ con modelo-er/, modelo-relacional/, scripts/ (01 a 05) y evidencias/.

Tamaño del grupo: 4 integrantes. Al no ser grupo de 5, no aplican los elementos adicionales (triggers, procedimiento y consulta extra); aun así el equipo entrega 8 triggers sobre 6 tablas, por encima del mínimo de 6.

Orden de ejecución: los scripts se ejecutan en orden 01 → 02 → 03 → 04 → 05 sobre una base de datos vacía de PostgreSQL. Para repetir la carga se vuelve a correr desde el 01, que borra y recrea las tablas.

Integrante	Usuario GitHub	Responsabilidad
Estefania Paredes Castañeda	tef333	Modelo ER, modelo relacional, script de creación de tablas, datos de actores/directores, staff y contratos
Jose Antonio Espin Molina	jespinn	Triggers, datos de salas, identidad de la empresa y logo
Santiago Rojas	Srojas-ga	Procedimientos almacenados, datos de tipos de boleta y ventas
Erika Gomez	Eri-02	Consultas SQL, datos de funciones y espectadores

1. Identidad de la empresa: Altamira Cine



Logo de Altamira Cine

Nombre: Altamira Cine. El nombre viene de la cueva de Altamira, en España, donde están algunas de las primeras imágenes pintadas por seres humanos: la idea de contar historias con imágenes es mucho más antigua que el cine. El logo muestra un bisonte, la figura más reconocida de esas pinturas, enmarcado en un arco que representa la entrada de la cueva y la pantalla de cine.

Ciudad: Bogotá, Colombia.

Enfoque de programación: cartelera comercial de estrenos combinada con cine colombiano y latinoamericano. Algunas funciones se convierten en eventos especiales con artistas invitados: estrenos con presencia del elenco, conversatorios con directores después de la proyección y presentaciones en vivo.

Modalidad de operación: presencial, en un solo complejo de salas. La venta de boletas se hace en taquilla, atendida por el personal, y en línea, sin intervención de un empleado. El pago en efectivo solo se acepta en taquilla.

Número de salas: 6.

Sala	Formato	Capacidad	Estado
Sala 1	2D	150	Activa
Sala 2	2D	150	Activa
Sala 3	3D	120	Activa
Sala VIP	VIP	60	Activa
Sala IMAX	IMAX	220	Activa
Sala 4DX	4DX	80	Mantenimiento

Capacidad estimada: 780 sillas en total, de las cuales 700 están en operación mientras la Sala 4DX sigue en mantenimiento.

Actores y directores registrados: 30, entre actores (14), directores (9) y artistas que cumplen ambos roles (7), de Colombia y otros países de Iberoamérica.

Particularidades del negocio:

- **Tarifas diferenciadas:** general, estudiante, niño, adulto mayor y VIP. El precio queda guardado en cada venta, así un cambio de tarifa no altera las ventas anteriores.
- **Programa de fidelización:** cada espectador acumula 1 punto por cada $1.000 de compra; los puntos se descuentan si la venta se anula.
- **Control de edad:** cada película tiene una clasificación (todo público, +7, +12, +15 y +18) que se verifica al vender la boleta.
- **Contratos con artistas:** cada contrato está ligado a una función específica, lo gestiona un administrador o coordinador del staff y tiene una vigencia definida.
- **Control de aforo:** nunca se venden más boletas que la capacidad de la sala, ni para funciones que ya empezaron o fueron canceladas.

2. Modelo entidad-relación

El modelo tiene las 8 entidades del enunciado. Ventas es el centro del negocio: une al espectador que compra, la función a la que asiste, el tipo de boleta y, cuando la venta es en taquilla, el miembro del staff que la registra. Las funciones se proyectan en una sala, y los contratos ligan a un actor o director con una función especial y con el staff que los gestiona.

Relación	Cardinalidad	Descripción
Salas – Funciones	1:N	Una sala aloja muchas funciones; cada función ocurre en una sala.
Funciones – Ventas	1:N	Una función genera muchas ventas.
Espectadores – Ventas	1:N	Un espectador realiza muchas compras.
Tipos_Boleta – Ventas	1:N	Cada venta es de un tipo de boleta.
Staff – Ventas	1:N (opcional)	El taquillero registra ventas de taquilla; las ventas en línea no tienen staff.
Actores_Directores – Contratos	1:N	Un artista puede firmar varios contratos.
Funciones – Contratos	1:N	Cada contrato es para una función especial.
Staff – Contratos	1:N	Un administrador o coordinador gestiona los contratos.

En la página siguiente está el diagrama completo

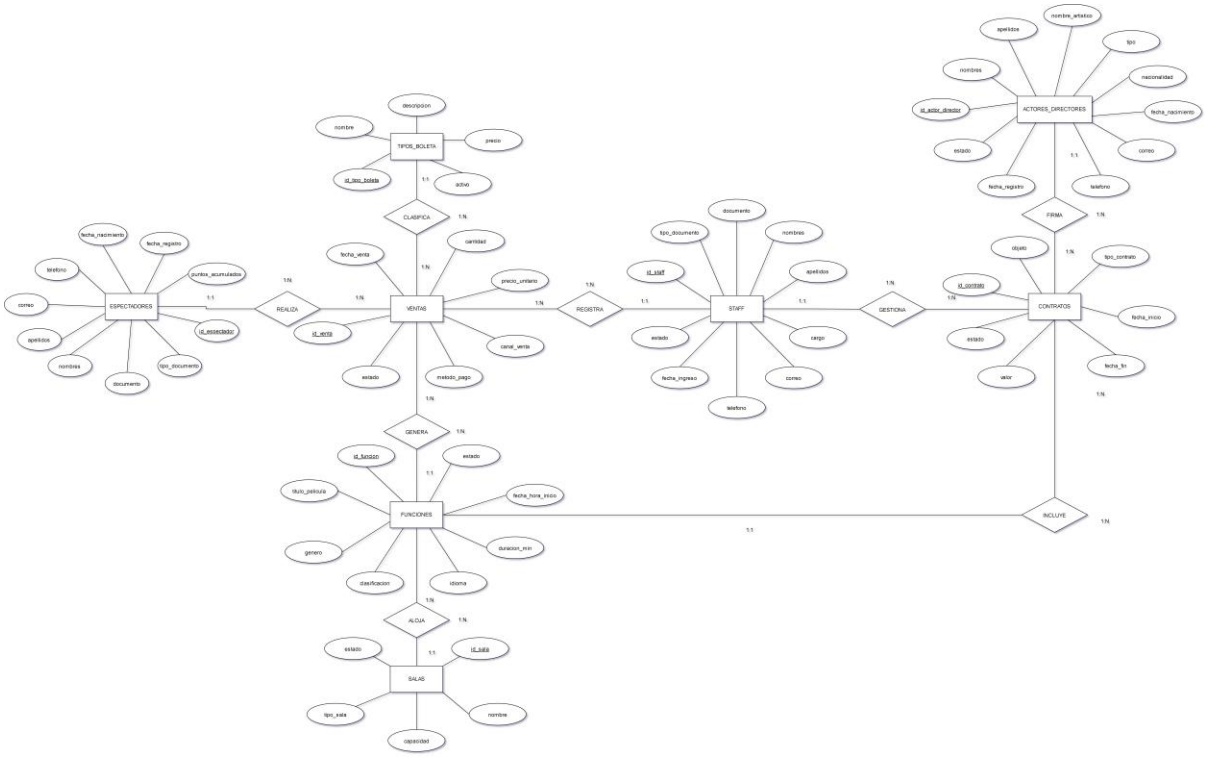


Figura 1. Modelo entidad-relación (avance1-postgresql/modelo-er/modelo-entidad-relacion.pdf)

3. Modelo relacional en 3FN

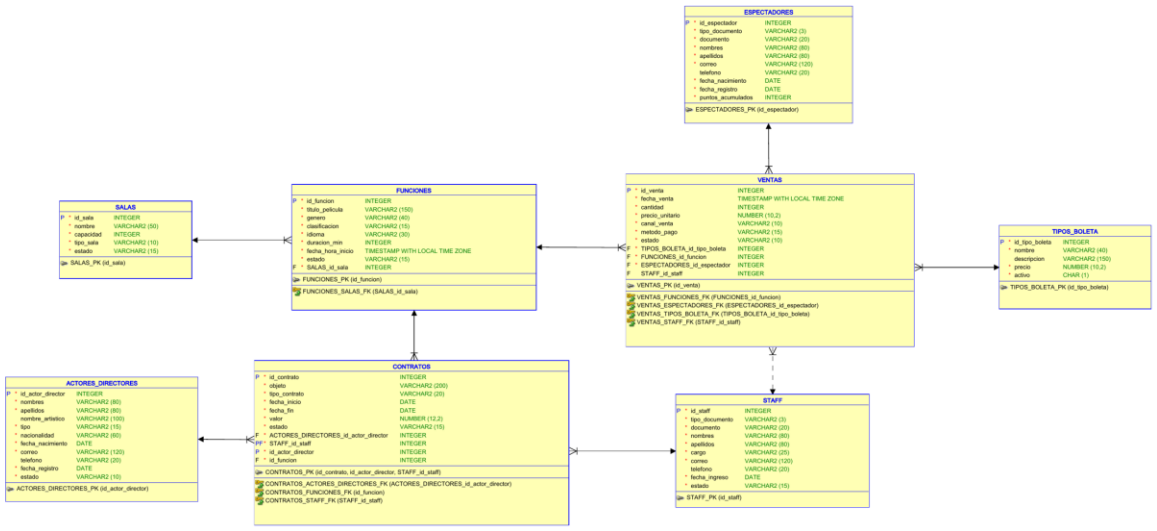


Figura 2. Modelo relacional normalizado (avance1-postgresql/modelo-relacional/modelo_relacional_3fn_final.pdf)

Cada tabla tiene llave primaria propia y las relaciones 1:N se resuelven con la llave foránea en la tabla hija. Se verificó la tercera forma normal:

- **1FN:** todos los atributos son atómicos (nombres y apellidos separados, un teléfono por registro).
- **2FN:** todas las llaves primarias son simples, por lo que no hay dependencias parciales.
- **3FN:** no hay atributos que dependan de otro atributo que no sea la llave. Los valores derivados no se almacenan: el total de una venta es cantidad × precio_unitario y la hora de fin de una función es inicio + duración.

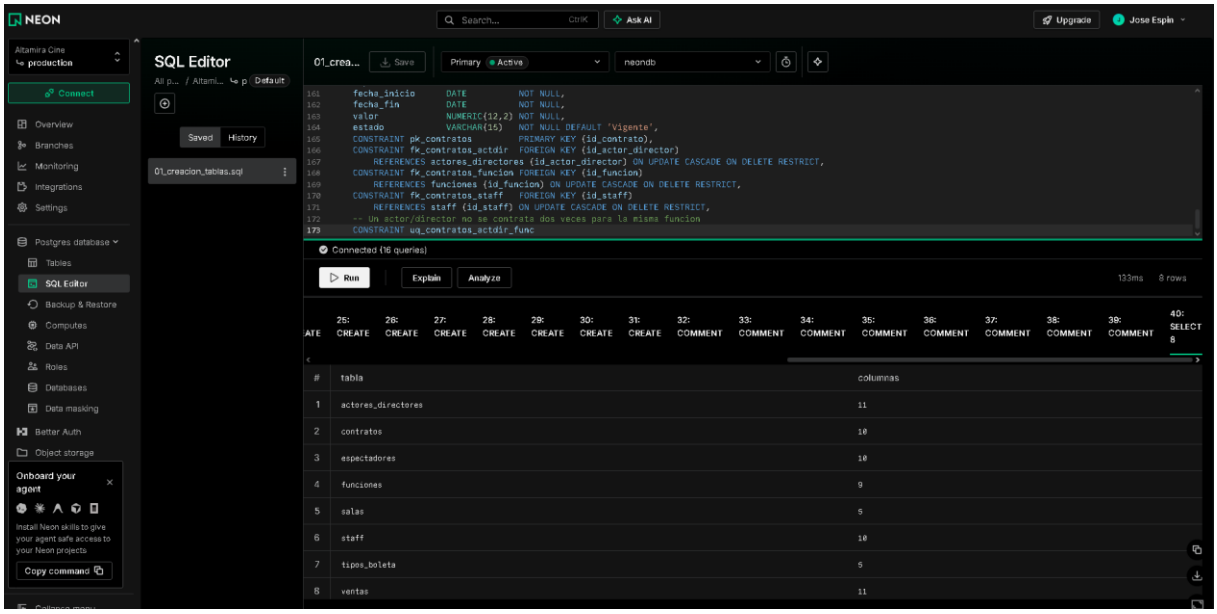
Decisión de diseño: los datos de la película (título, género, clasificación, duración) se guardan en Funciones porque el alcance fija exactamente 8 entidades y no incluye un catálogo de películas. El equipo identificó que esto repite datos cuando una película tiene varias funciones; en un avance posterior se resolvería con una entidad Películas.

Nota: precio_unitario en Ventas no es redundante con el precio de Tipos_Boleta: guarda el precio vigente al momento de la venta, para que un cambio de tarifa no altere el histórico.

4. Script de creación (01_creacion_tablas.sql)

Crea las 8 tablas con llaves primarias (IDENTITY), llaves foráneas con ON DELETE RESTRICT, restricciones UNIQUE, NOT NULL, CHECK, valores por defecto e índices sobre las llaves foráneas y columnas de búsqueda frecuente. Las reglas de una sola fila quedan como CHECK; las que necesitan consultar otras tablas se resuelven con triggers (sección 6).

Tabla	Restricciones principales
salas	UNIQUE nombre; CHECK capacidad entre 1 y 1000; tipo 2D/3D/IMAX/VIP/4DX; estado Activa/Mantenimiento/Inactiva
tipos_boleta	UNIQUE nombre; CHECK precio ≥ 0; activo BOOLEAN
actores_directores	UNIQUE correo y (nombres, apellidos, fecha_nacimiento); CHECK tipo, estado y formato de correo
staff	UNIQUE (tipo_documento, documento) y correo; CHECK cargo, estado y formato de correo
espectadores	UNIQUE (tipo_documento, documento) y correo; CHECK puntos ≥ 0 y formato de correo
funciones	FK a salas; UNIQUE (sala, hora de inicio); CHECK clasificación, duración y estado
contratos	FK a actores, funciones y staff; UNIQUE (artista, función); CHECK fecha_fin ≥ fecha_inicio, valor > 0, tipo y estado
ventas	FK a funciones, espectadores, tipos_boleta y staff; CHECK cantidad 1–10, canal, método y estado; taquilla con staff y en línea sin staff; efectivo solo en taquilla



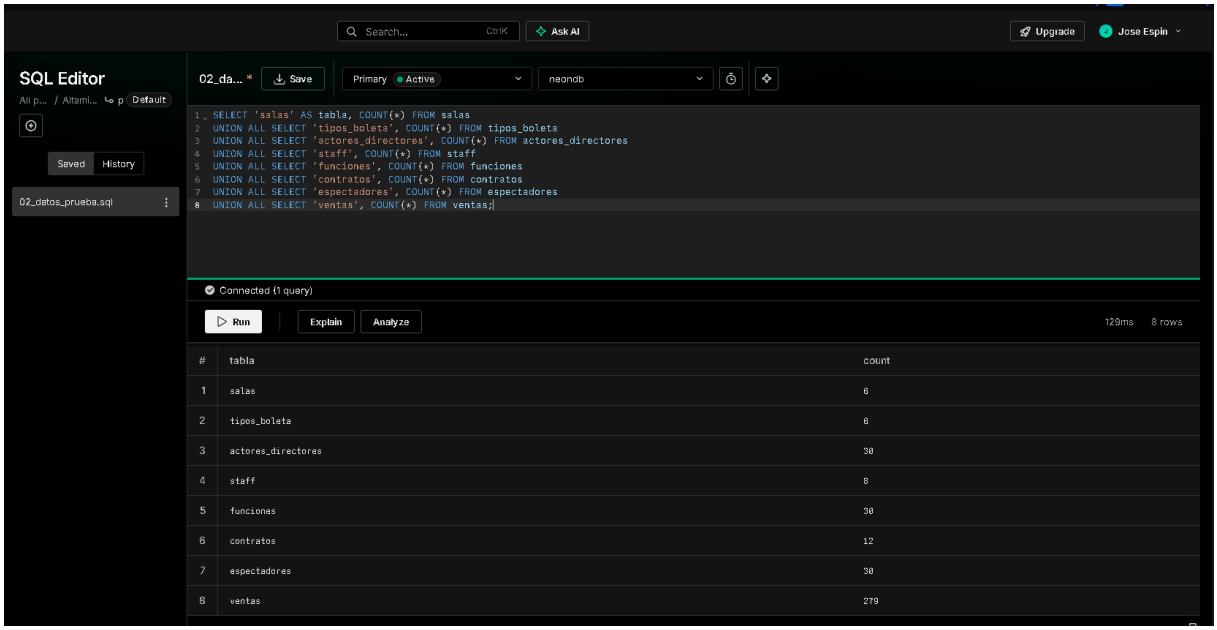
Evidencia: ejecución de 01_creacion_tablas.sql; la consulta final lista las 8 tablas y su número de columnas (ejecución en PostgreSQL, Neon)

5. Datos de prueba (02_datos_prueba.sql)

Los datos se cargan en el orden de las llaves foráneas, con id explícitos para que las secciones de cada integrante se puedan referenciar. Al final se sincronizan los contadores de id. Se cumple el mínimo de 30 registros en actores/directores, espectadores, ventas y funciones.

Tabla	Registros	Detalle
salas	6	5 activas y 1 en mantenimiento
tipos_boleta	6	5 activos y 1 promoción retirada

actores_directores	30	14 actores, 9 directores, 7 actor/director
staff	8	Administrador, coordinador, taquilleros, proyeccionista, acomodador y servicio al cliente
funciones	30	16 finalizadas, 12 programadas a futuro y 2 canceladas
contratos	12	5 vigentes, 5 finalizados y 2 cancelados
espectadores	30	Incluye menores de edad para probar el control por clasificación
ventas	279	261 en taquilla y en línea pagadas y 18 anuladas



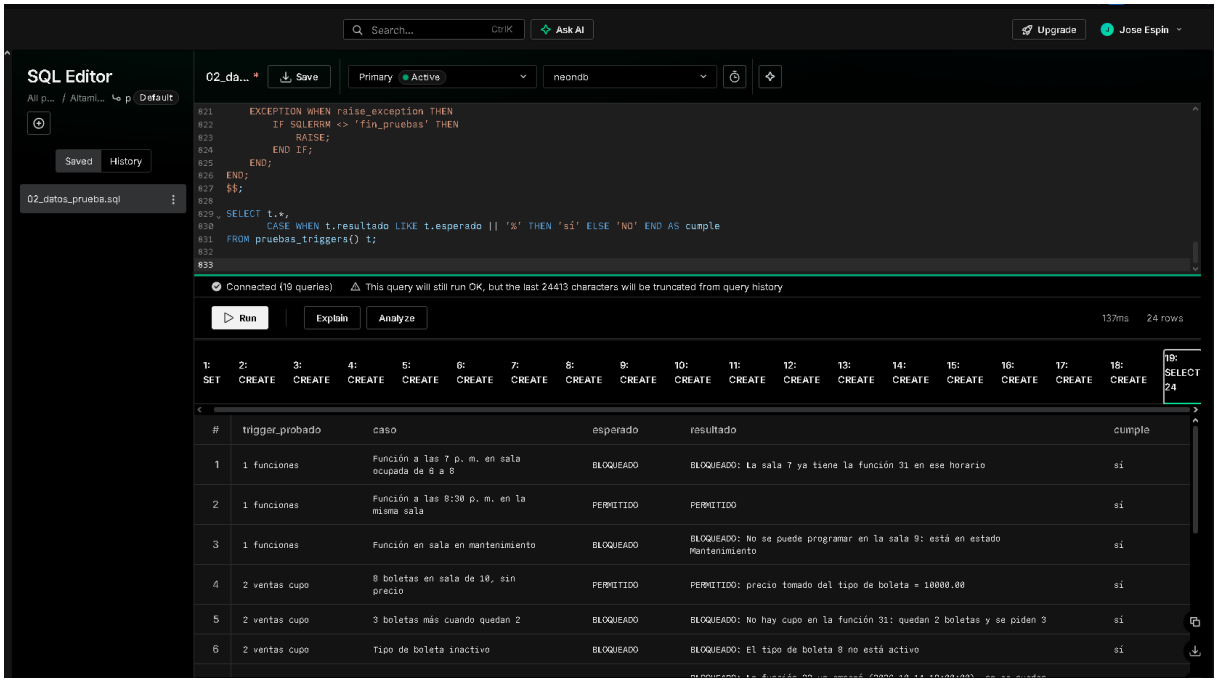
Evidencia: conteo de registros por tabla después de ejecutar 02_datos_prueba.sql (ejecución en PostgreSQL, Neon)

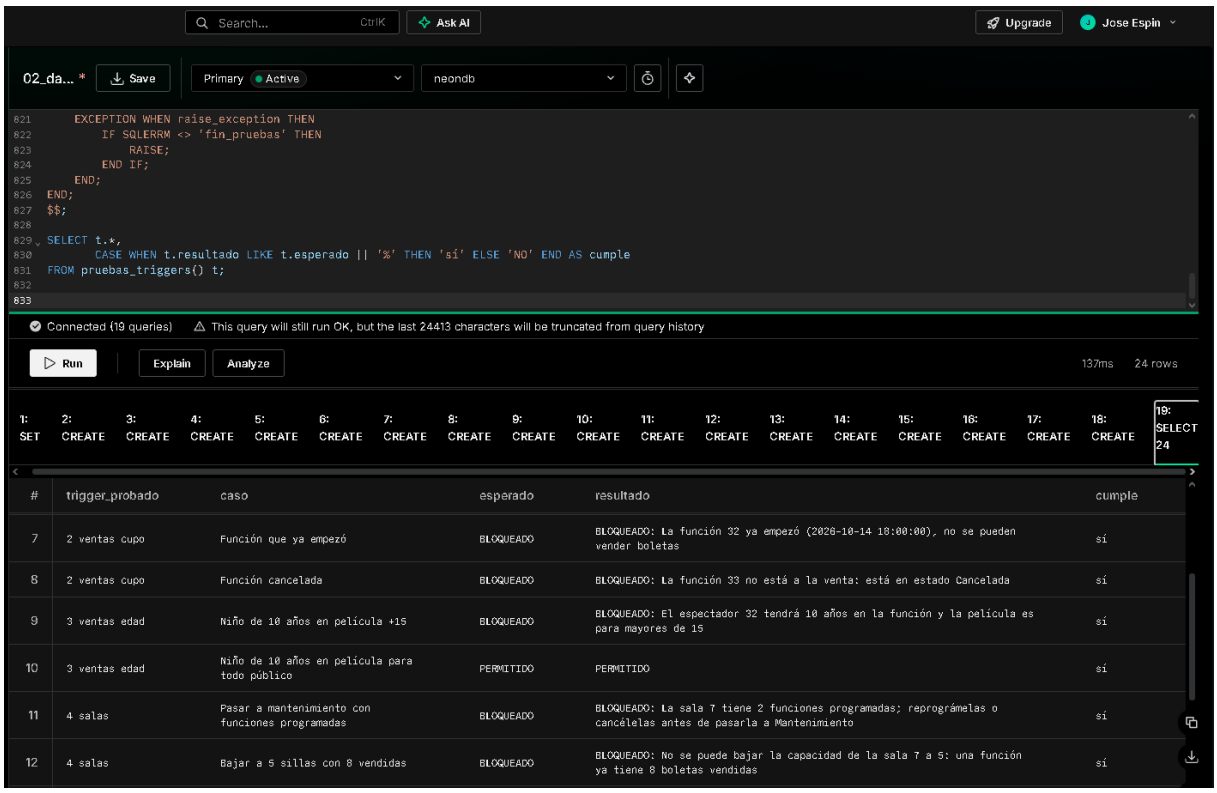
6. Triggers (03_triggers.sql)

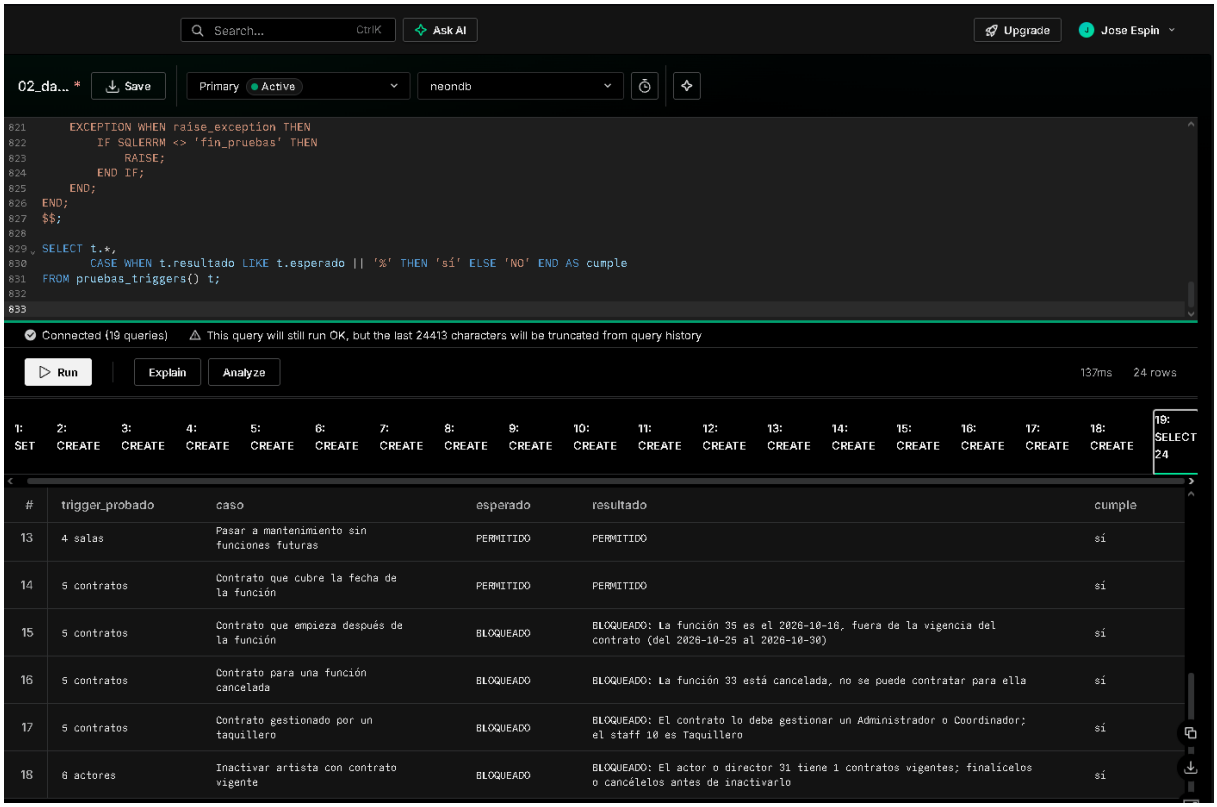
8 triggers sobre 6 de las 8 tablas: funciones, ventas, salas, contratos, actores_directores y espectadores. Cada trigger cubre una regla que un CHECK no puede validar, porque necesita consultar otras filas u otras tablas, o corregir el dato antes de guardarlo.

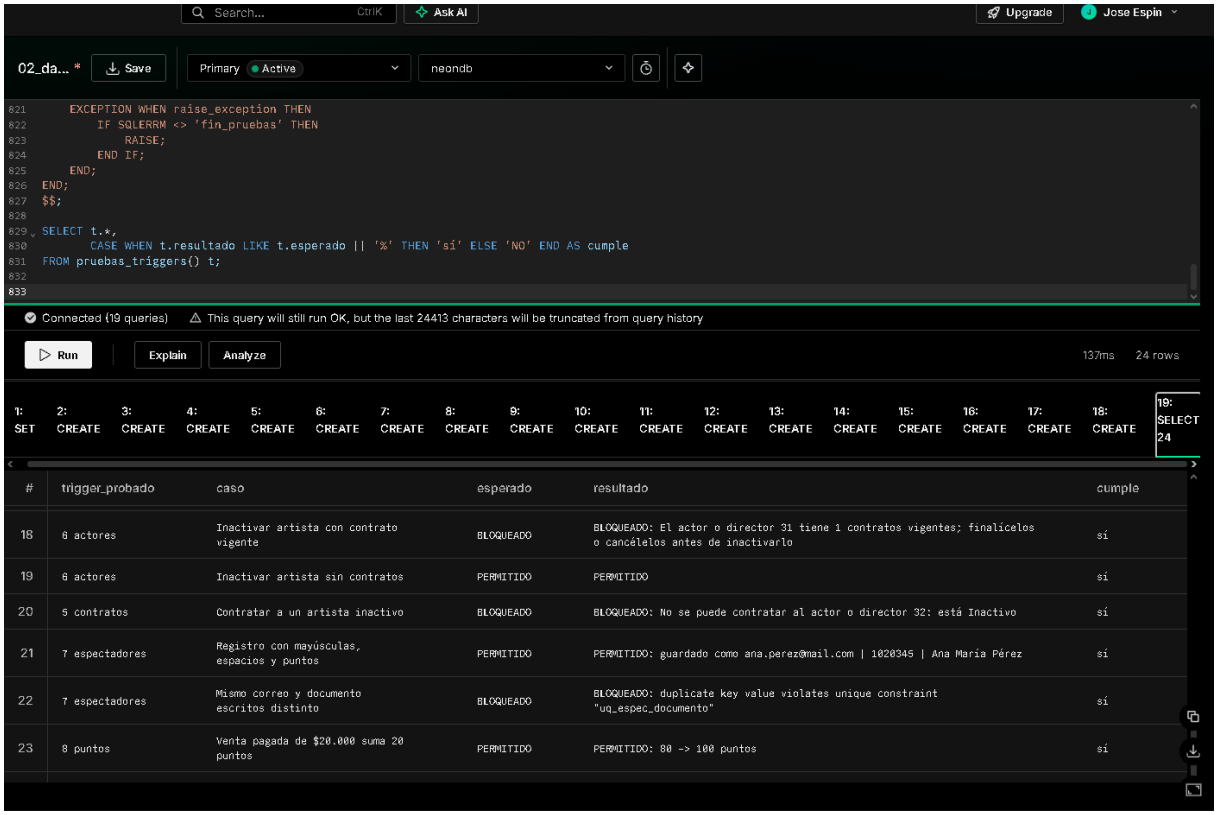
#	Trigger y tabla	Regla de negocio que protege	Por qué es trigger
1	trg_funciones_validar_programacion (funciones, BEFORE)	Una función programada solo va en una sala activa y no se cruza en horario con otra de la misma sala.	El cruce depende de las otras funciones; el UNIQUE solo detecta la misma hora exacta.
2	trg_ventas_validar_cupo (ventas, BEFORE)	No se vende por encima de la capacidad, ni para funciones que empezaron o se cancelaron, ni con tipos de boleta inactivos. Toma el precio vigente si falta.	El cupo es capacidad menos lo vendido; usa FOR UPDATE para ventas simultáneas.
3	trg_ventas_validar_edad (ventas, BEFORE)	El espectador debe tener la edad mínima de la clasificación el día de la función.	La edad está en espectadores y la clasificación en funciones.
4	trg_salas_validar_cambios (salas, BEFORE)	Una sala no sale de servicio con funciones futuras ni baja su capacidad por debajo de lo vendido.	Depende de funciones y ventas.
5	trg_contratos_validar (contratos, BEFORE)	El contrato lo gestiona un administrador o coordinador activo, con un artista activo, para una función no cancelada y dentro de su vigencia.	Cargo, estado del artista y fecha de la función están en otras tablas.
6	trg_actores_validar_inactivacion (actores_directores, BEFORE)	No se inactiva a un artista con contratos vigentes.	Los contratos están en otra tabla.
7	trg_espectadores_normalizar (espectadores, BEFORE)	Correo en minúsculas, documento sin puntos y nombres con mayúscula inicial, para que los UNIQUE detecten duplicados.	Un CHECK solo rechaza; el trigger corrige el dato.
8	trg_ventas_acumular_puntos (ventas, AFTER)	Suma 1 punto por cada $1.000 de una venta pagada y los ajusta si se anula, borra o cambia.	El saldo está en espectadores; es AFTER para sumar solo ventas ya validadas y guardadas.

Verificación: al final del script, la función pruebas_triggers() crea sus propios datos, intenta para cada trigger un caso que debe bloquearse y otro que debe pasar, y deshace todo al terminar. Son 24 casos y en todos la columna "cumple" dice sí.


Search... Ctrl K
Ask AI
Upgrade
Jose Espin
SQL Editor
02_da... *
Save
Primary Active
neondb
Saved
History
02_datos_pruebas.sql
EXCEPTION WHEN raise_exception THEN
IF SQLERRM <> 'fin_pruebas' THEN
RAISE;
END IF;
END;
END;
$$;
SELECT t.*,
CASE WHEN t.resultado LIKE t.esperado || '%' THEN 'sí' ELSE 'NO' END AS cumple
FROM pruebas_triggers() t;
Connected (19 queries)
This query will still run OK, but the last 24413 characters will be truncated from query history
Run
Explain
Analyze
137ms
24 rows
1: SET
2: CREATE
3: CREATE
4: CREATE
5: CREATE
6: CREATE
7: CREATE
8: CREATE
9: CREATE
10: CREATE
11: CREATE
12: CREATE
13: CREATE
14: CREATE
15: CREATE
16: CREATE
17: CREATE
18: CREATE
19: SELECT 24
trigger_probado caso esperado resultado cumple
1 1 funciones Función a las 7 p. m. en sala ocupada de 8 a 8 BLOQUEADO BLOQUEADO: La sala 7 ya tiene la función 31 en ese horario sí
2 1 funciones Función a las 8:30 p. m. en la misma sala PERMITIDO PERMITIDO sí
3 1 funciones Función en sala en mantenimiento BLOQUEADO BLOQUEADO: No se puede programar en la sala 9: está en estado Mantenimiento sí
4 2 ventas cupo 8 boletas en sala de 10, sin precio PERMITIDO PERMITIDO: precio tomado del tipo de boleta = 10000.00 sí
5 2 ventas cupo 3 boletas más cuando quedan 2 BLOQUEADO BLOQUEADO: No hay cupo en la función 31: quedan 2 boletas y se piden 3 sí
6 2 ventas cupo Tipo de boleta inactivo BLOQUEADO BLOQUEADO: El tipo de boleta 8 no está activo sí


Search... Ctrl K
Ask AI
Upgrade
Jose Espin
02_da... *
Save
Primary Active
neondb
EXCEPTION WHEN raise_exception THEN
IF SQLERRM <> 'fin_pruebas' THEN
RAISE;
END IF;
END;
END;
$$;
SELECT t.*,
CASE WHEN t.resultado LIKE t.esperado || '%' THEN 'sí' ELSE 'NO' END AS cumple
FROM pruebas_triggers() t;
Connected (19 queries)
This query will still run OK, but the last 24413 characters will be truncated from query history
Run
Explain
Analyze
137ms
24 rows
1: SET
2: CREATE
3: CREATE
4: CREATE
5: CREATE
6: CREATE
7: CREATE
8: CREATE
9: CREATE
10: CREATE
11: CREATE
12: CREATE
13: CREATE
14: CREATE
15: CREATE
16: CREATE
17: CREATE
18: CREATE
19: SELECT 24
trigger_probado caso esperado resultado cumple
7 2 ventas cupo Función que ya empezó BLOQUEADO BLOQUEADO: La función 32 ya empezó (2028-10-14 18:00:00), no se pueden vender boletas sí
8 2 ventas cupo Función cancelada BLOQUEADO BLOQUEADO: La función 33 no está a la venta: está en estado Cancelada sí
9 3 ventas edad Niño de 10 años en película +15 BLOQUEADO BLOQUEADO: El espectador 32 tendrá 10 años en la función y la película es para mayores de 15 sí
10 3 ventas edad Niño de 10 años en película para todo público PERMITIDO PERMITIDO sí
11 4 salas Pasar a mantenimiento con funciones programadas BLOQUEADO BLOQUEADO: La sala 7 tiene 2 funciones programadas; reprográmelas o cancélelas antes de pasarla a Mantenimiento sí
12 4 salas Bajar a 5 sillas con 8 vendidas BLOQUEADO BLOQUEADO: No se puede bajar la capacidad de la sala 7 a 5: una función ya tiene 8 boletas vendidas sí





Evidencia: resultado de pruebas_triggers(): 24 casos, todos con cumple = sí (ejecución en PostgreSQL, Neon)

7. Procedimientos almacenados (04_procedimientos.sql)

Los tres procedimientos manejan errores con EXCEPTION y RAISE: los errores de negocio y de los triggers se propagan con su mensaje original, y cualquier otro error se reporta con su código SQLSTATE.

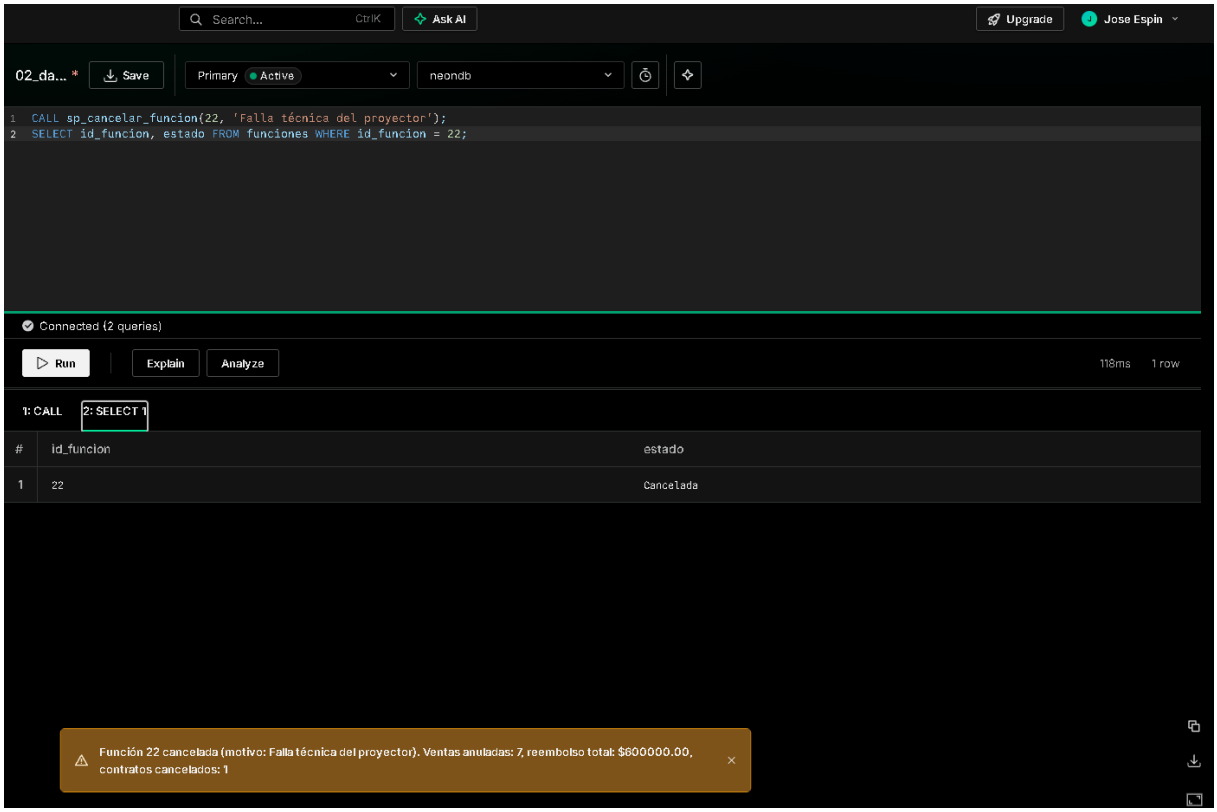
7.1 sp_cancelar_funcion · cálculo y transacción

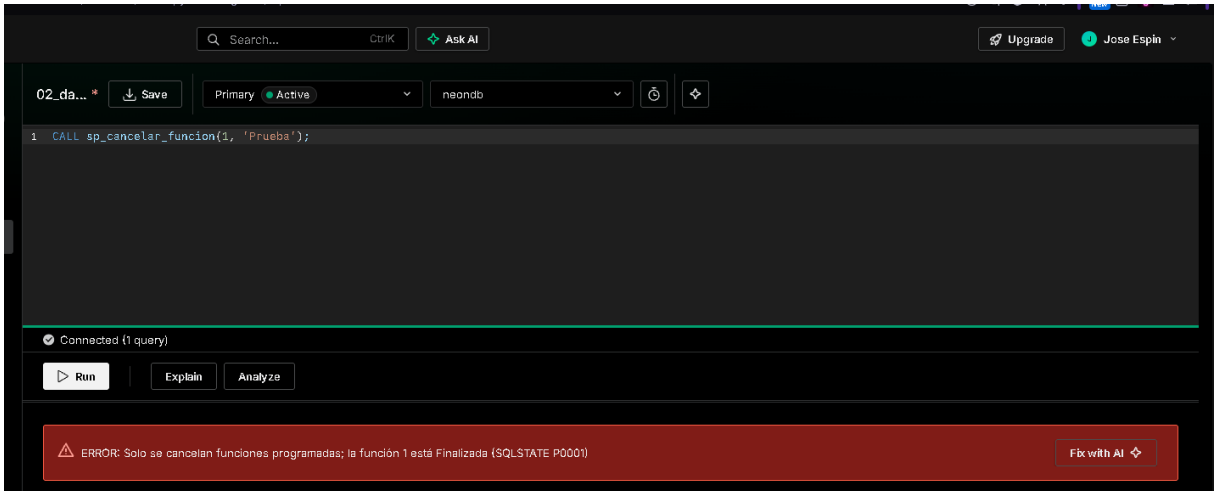
Propósito: cancela una función programada y todo lo que depende de ella en una sola transacción: calcula el valor a reembolsar, anula las ventas pagadas, cancela los contratos vigentes y marca la función como Cancelada. Si algo falla, no queda nada a medias.

Validaciones: la función debe existir, estar programada y no haber empezado. Bloquea la fila con FOR UPDATE. Al anular las ventas, el trigger 8 descuenta los puntos de los espectadores.

```
CALL sp_cancelar_funcion(22, 'Falla técnica del proyector');
SELECT id_funcion, estado FROM funciones WHERE id_funcion = 22;
```

Resultado: la función 22 quedó Cancelada, con 7 ventas anuladas, $600.000 de reembolso y 1 contrato cancelado. El segundo intento, sobre la función 1 que ya está Finalizada, se rechaza con el mensaje de negocio.





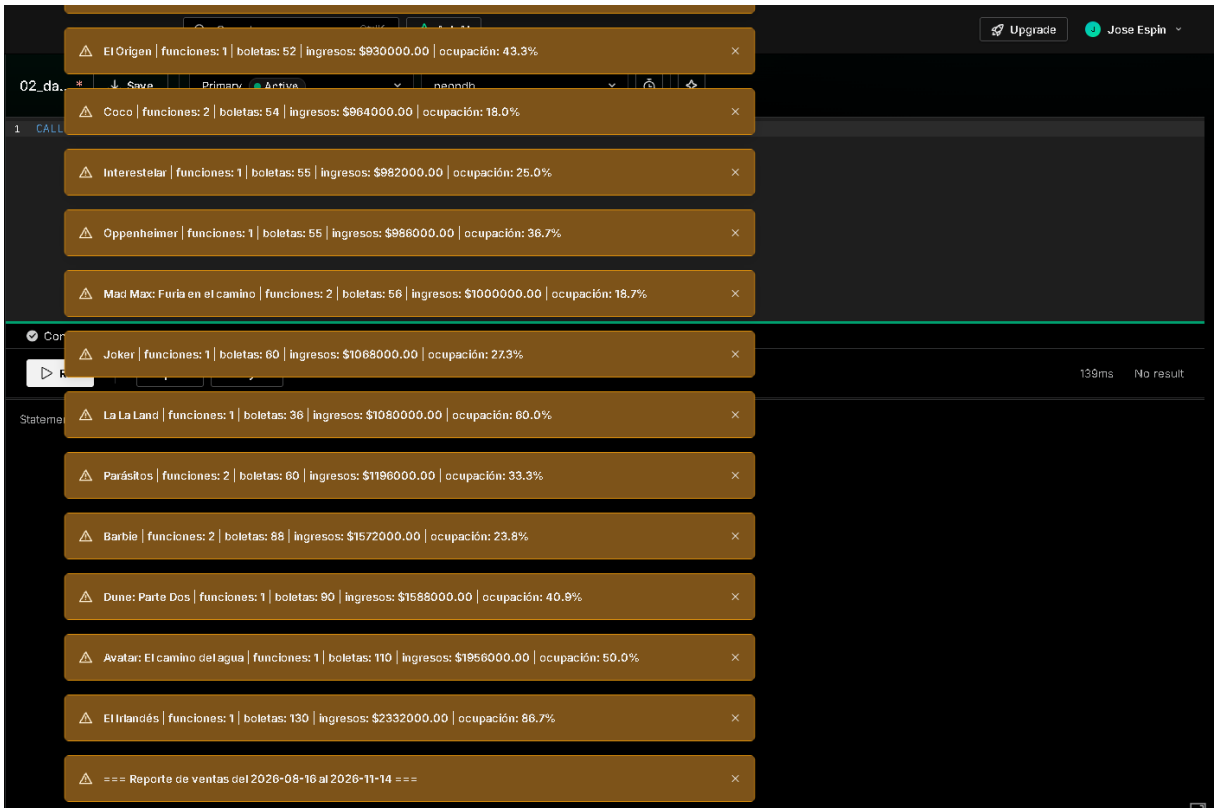
Evidencia: cancelación exitosa de la función 22 y rechazo al intentar cancelar una función finalizada (ejecución en PostgreSQL, Neon)

7.2 sp_reporte_ventas · reporte con agregación

Propósito: por película, en un rango de fechas, muestra número de funciones, boletas vendidas, ingresos y porcentaje de ocupación, más un total general. No cuenta funciones canceladas ni ventas anuladas.

Validaciones: las dos fechas son obligatorias y la inicial no puede ser posterior a la final.

```
CALL sp_reporte_ventas(CURRENT_DATE - 60, CURRENT_DATE + 30);
```



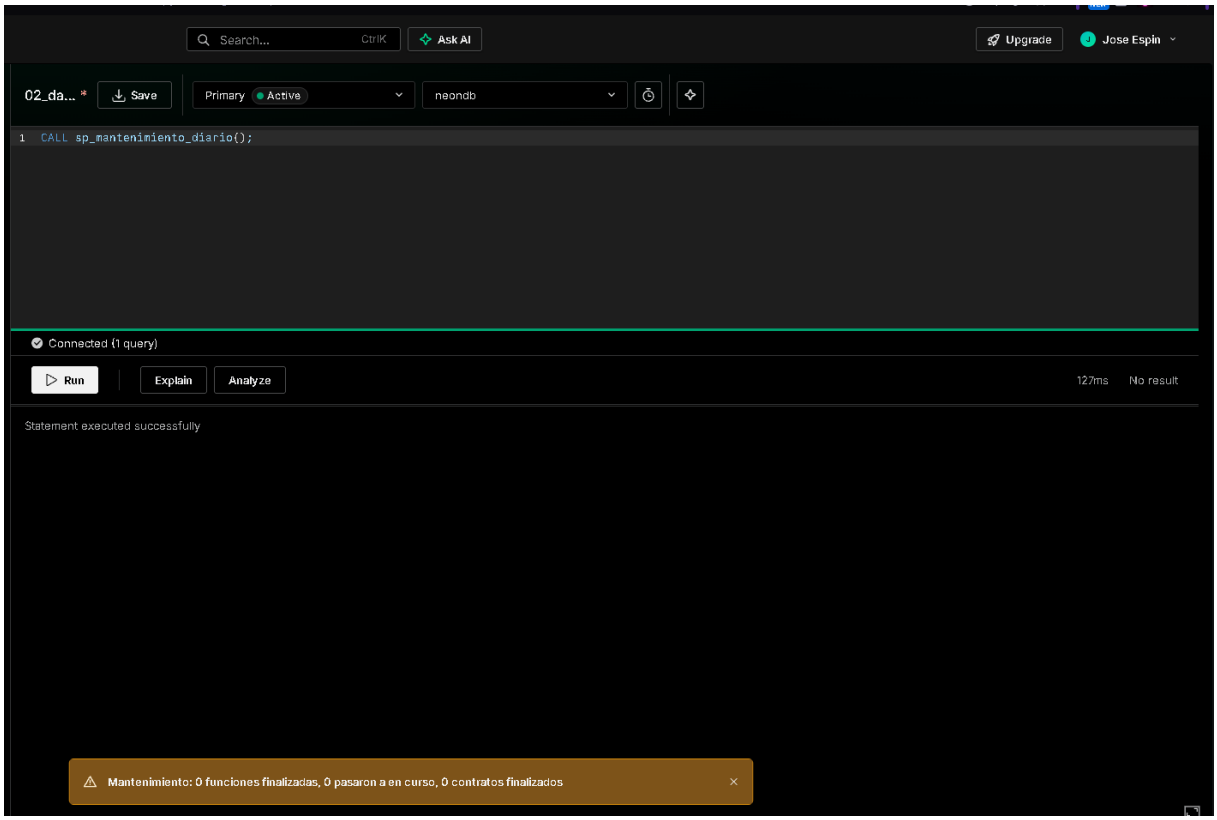
Evidencia: reporte de ventas por película con funciones, boletas, ingresos y ocupación (ejecución en PostgreSQL, Neon)

7.3 sp_mantenimiento_diario · mantenimiento y operación

Propósito: pensado para ejecutarse cada día: pasa a Finalizada las funciones cuya hora de fin ya pasó, a En curso las que ya empezaron, y a Finalizado los contratos vencidos.

```
CALL sp_mantenimiento_diario();
```

Resultado: en esta ejecución no hubo cambios (0 en cada caso) porque los datos de prueba ya tenían los estados al día. El procedimiento está pensado para correr cada día y actualizar lo que haya vencido desde la última ejecución.



Evidencia: ejecución de sp_mantenimiento_diario (ejecución en PostgreSQL, Neon)

8. Consultas SQL (05_consultas.sql)

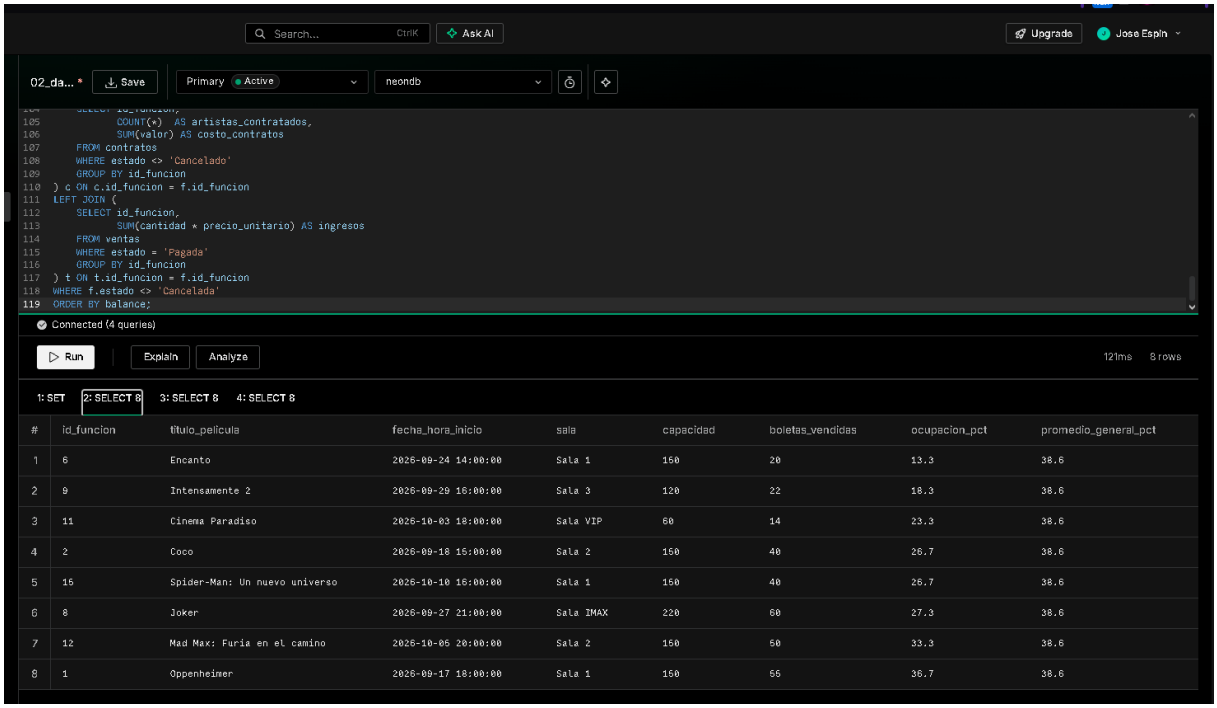
Las tres consultas solo leen datos. Cuentan ventas pagadas y excluyen funciones canceladas.

8.1 Funciones con ocupación por debajo del promedio

Propósito de negocio: detectar las funciones ya realizadas que se llenaron poco, para decidir si conviene programar menos funciones de esa película u horario, usar una sala más pequeña o lanzar promociones.

Técnicas: JOIN funciones–salas, LEFT JOIN a ventas para incluir funciones sin ventas, SUM con GROUP BY, CTE y subconsulta escalar con AVG.

Resultado: 8 funciones finalizadas quedaron por debajo del promedio de ocupación (38,6 %); la más baja es Encanto, con 13,3 %.



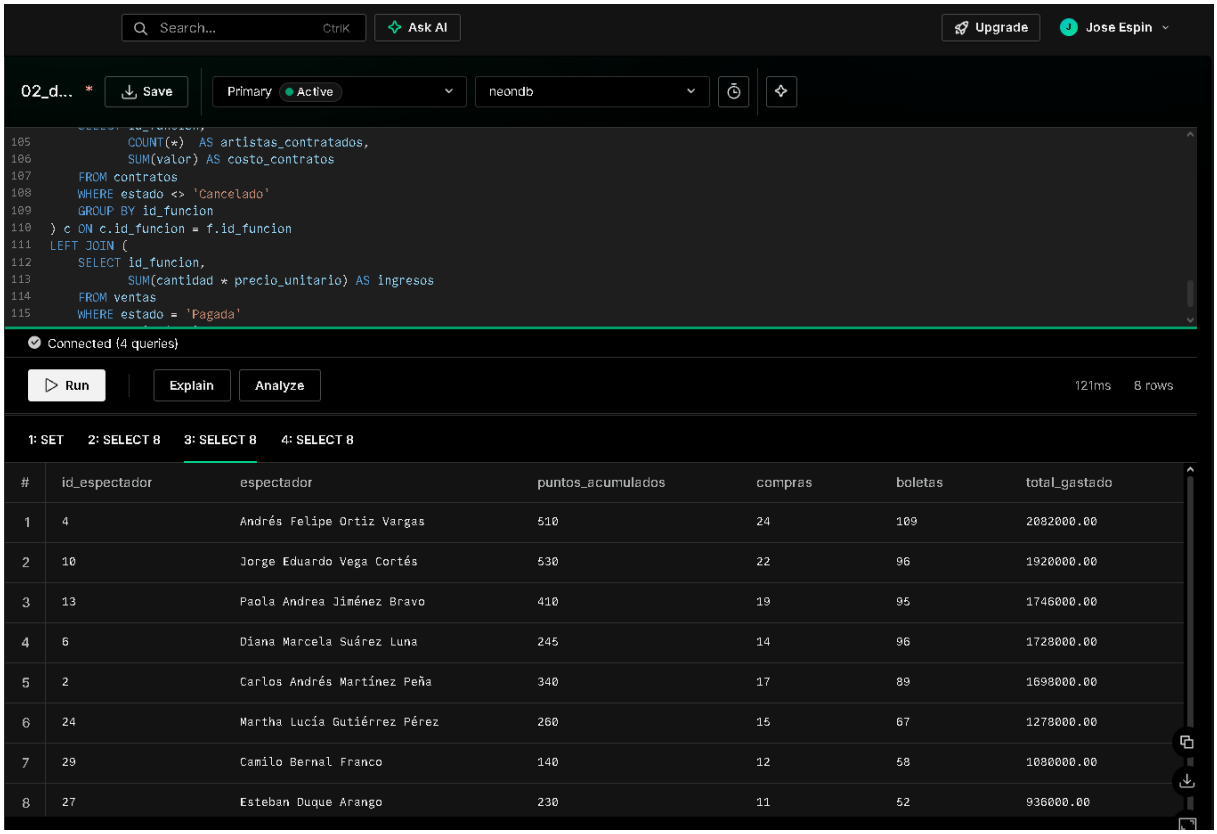
Evidencia: consulta 1, funciones con ocupación por debajo del promedio (ejecución en PostgreSQL, Neon)

8.2 Espectadores que gastan más que el promedio

Propósito de negocio: identificar a los mejores clientes para el programa de fidelización: descuentos, preventas e invitaciones a eventos.

Técnicas: JOIN ventas–espectadores, SUM y COUNT con GROUP BY, y HAVING con una subconsulta que calcula el gasto promedio por espectador.

Resultado: 8 espectadores gastan más que el promedio; el primero acumula $2.082.000 en 24 compras.



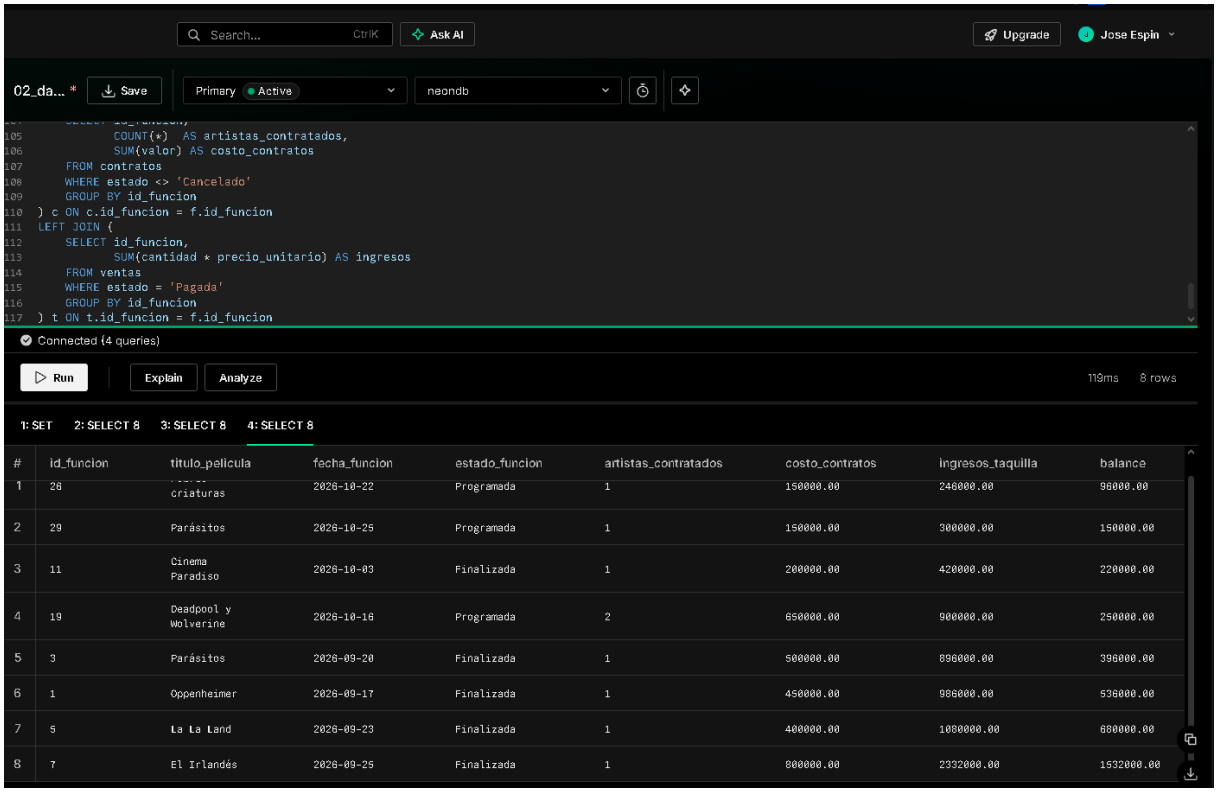
Evidencia: consulta 2, espectadores que gastan más que el promedio (ejecución en PostgreSQL, Neon)

8.3 Costo de contratos vs. ingresos de taquilla

Propósito de negocio: saber si las funciones especiales con actores o directores contratados se pagan solas con la taquilla o dan pérdida.

Técnicas: JOIN de tres fuentes y subconsultas en el FROM que agregan contratos y ventas por separado, para que los valores no se multipliquen al cruzar filas.

Resultado: las 8 funciones especiales vigentes tienen balance positivo: la taquilla cubre el costo de los artistas. La función 22 no aparece porque se canceló en la prueba del procedimiento 7.1.



Evidencia: consulta 3, costo de contratos frente a ingresos de taquilla (ejecución en PostgreSQL, Neon)

Conclusión

El núcleo transaccional de Altamira Cine queda con 8 tablas en 3FN, datos de prueba coherentes entre sí, 8 triggers que protegen las reglas del negocio en 6 tablas, 3 procedimientos con manejo de excepciones y 3 consultas de análisis. El control de integridad está repartido en casi todas las tablas, como pide el avance: las reglas de una fila viven en restricciones CHECK y las que cruzan tablas en triggers y procedimientos.